# Глава 8

# Потоки данных

В этой главе...


- 8.1. От итерации к потоковым операциям
- 8.2. Создание потока данных
- 8.3. Методы `filter()`, `map()` и `flatMap()`
- 8.4. Извлечение и соединение потоков данных
- 8.5. Другие потоковые преобразования
- 8.6. Простые методы сведения
- 8.7. Тип `Optional`
- 8.8. Накопление результатов
- 8.9. Накопление результатов в отображениях
- 8.10. Группирование и разделение
- 8.11. Нисходящие коллекторы
- 8.12. Операции сведения
- 8.13. Потоки данных примитивных типов
- 8.14. Параллельные потоки данных
- Упражнения

Потоки данных обеспечивают представление данных, позволяющее указать вычисления на более высоком концептуальном уровне, чем коллекции. С помощью потока данных можно указать, что и как именно требуется сделать с данными, а планирование операций предоставить конкретной реализации. Допустим, что требуется вычислить среднее некоторого свойства. С этой целью указывается источник данных и свойство, а средствами библиотеки потоков данных можно оптимизировать вычисление, используя, например, несколько потоков исполнения для расчета сумм, подсчета и объединения результатов.

Основные положения этой главы приведены ниже.

1. Итераторы подразумевают конкретную методику обхода и препятствуют организации эффективного параллельного выполнения.
2. Потоки данных можно создавать из коллекций, массивов, генераторов или итераторов.
3. Для отбора элементов из потока данных служит метод `filter()`, а для их преобразования — метод `map()`.
4. Другие операции преобразования потоков данных реализуются методами `limit()`, `distinct()` и `sorted()`.
5. Для получения результата из потока данных служат операции сведения, реализуемые, в частности, методами `count()`, `max()`, `min()`, `findFirst()` и `findAny()`. Некоторые из этих методов возвращают необязательное значение типа `Optional`.
6. Тип `Optional` специально предназначен в качестве надежной альтернативы обработке пустых значений `null`. Для надежного применения этого типа служат методы `ifPresent()` и `orElse()`.
7. Результаты, получаемые из потоков данных, можно накапливать в коллекциях, массивах, символьных строках или отображениях.
8. Методы `groupingBy()` и `partitioningBy()` из класса `Collectors` позволяют разделять содержимое потоков данных на группы и получать результат для каждой группы в отдельности.
9. Для примитивных типов `int`, `long` и `double` имеются специализированные потоки данных.
10. Параллельные потоки данных автоматически распараллеливают потоковые операции.

## 8.1. От итерации к потоковым операциям

Для обработки коллекции обычно требуется перебрать ее элементы и выполнить над ними некоторую операцию. Допустим, что требуется подсчитать все длинные слова в книге. Сначала организуем их вывод списком следующим образом:

```
String contents = new String(Files.readAllBytes(
  Paths.get("alice.txt")), StandardCharsets.UTF_8);
```

```
    // прочитать текст из файла в символьную строку
List<String> words = Arrays.asList(contents.split("\\PL+"));
    // разбить полученную символьную строку на слова;
    // небуквенные символы считаются разделителями
```

А теперь можно перебрать слова таким образом:

```
int count = 0;
for (String w : words) {
  if (w.length() > 12) count++;
}
```

Ниже показано, каким образом аналогичная операция осуществляется с помощью потоков данных.

```
long count = words.stream()
    .filter(w -> w.length() > 12)
    .count();
```

В последнем случае не нужно искать в цикле наглядного подтверждения операций фильтрации и подсчета слов. Сами имена методов свидетельствуют о том, что именно предполагается сделать в коде. Более того, если в цикле во всех подробностях предписывается порядок выполнения операций, то в потоке данных операции можно планировать как угодно, при условии, что будет достигнут правильный результат.

Достаточно заменить метод `stream()` на метод `parallelStream()`, чтобы организовать средствами библиотеки потоков данных параллельное выполнение операций фильтрации и подсчета слов, как показано ниже.

```
long count = words.parallelStream()
    .filter(w -> w.length() > 12)
    .count();
```

Потоки данных действуют по принципу "что, а не как делать". В рассматриваемом здесь примере мы описываем, что нужно сделать: получить длинные слова и подсчитать их. При этом мы не указываем, в каком порядке или потоке исполнения это должно произойти. Напротив, в упомянутом выше цикле точно указывается порядок организации вычислений, а следовательно, исключается всякая возможность для оптимизации.

На первый взгляд поток данных похож на коллекцию, поскольку он позволяет преобразовывать и извлекать данные. Но у потока данных имеются следующие существенные отличия:

1. Поток данных не сохраняет свои элементы. Они могут храниться в основной коллекции или формироваться по требованию.
2. Потоковые операции не изменяют их источник. Например, метод `filter()` не удаляет элементы из нового потока данных, но выдает новый поток, в котором они отсутствуют.
3. Потоковые операции выполняются *по требованию*, когда это возможно. Это означает, что они не выполняются до тех пор, пока не потребуется их результат. Так, если требуется подсчитать только пять длинных слов вместо всех слов,

метод filter() прекратит фильтрацию после пятого совпадения. Следовательно, потоки данных могут быть бесконечными!

Вернемся к предыдущему примеру, чтобы рассмотреть его подробнее. Методы stream() и parallelStream() выдают *поток данных* для списка слов words. А метод filter() возвращает другой поток данных, содержащий только те слова, длина которых больше 12 букв. И наконец, метод count() сводит этот поток данных в конечный результат.

Такая последовательность операций весьма характерна для обращения с потоками данных. Конвейер операций организуется в следующие три стадии.

1. Создание потока данных.
2. Указание *промежуточных операций* для преобразования исходного потока данных в другие потоки, возможно, в несколько этапов.
3. Выполнение *оконечной операции* для получения результата. Эта операция принуждает к выполнению по требованию тех операций, которые ей предшествуют. А впоследствии поток данных может больше не понадобиться.

В рассматриваемом здесь примере поток данных был создан методом stream() или parallelStream(). Метод filter() преобразовал его, а метод count() выполнил оконечную операцию.

В следующем разделе будет показано, как создается поток данных. В трех последующих разделах рассматриваются потоковые преобразования, а в пяти следующих за ними разделах — оконечные операции.

## 8.2. Создание потока данных

Как было показано выше, любую коллекцию можно преобразовать в поток данных методом stream() из интерфейса Collection. Если же вместо коллекции имеется массив, то для этой цели служит метод Stream.of(), как показано ниже.

```
Stream<String> words = Stream.of(contents.split("\\PL+"));
    // Метод split() возвращает массив типа String[]
```

У метода of() имеются аргументы переменной длины, и поэтому поток данных можно построить из любого количества аргументов, как показано ниже. А для создания потока данных из части массива служит метод Arrays.stream(array, from, to).

```
Stream<String> song = Stream.of("gently", "down", "the", "stream");
```

Чтобы создать поток данных без элементов, достаточно вызвать статический метод Stream.empty() следующим образом:

```
Stream<String> silence = Stream.empty();
    // Обобщенный тип <String> выводится автоматически;
    // что равнозначно вызову Stream.<String>empty()
```

Для создания бесконечных потоков данных в интерфейсе Stream имеются два статических метода. В частности, метод generate() принимает функцию без аргументов

(а формально — объект функционального интерфейса `Supplier<T>`; см. раздел 3.6.2). Всякий раз, когда требуется потоковое значение, эта функция вызывается для получения данного значения. Например, поток постоянных значений можно получить следующим образом:

```
Stream<String> echos = Stream.generate(() -> "Echo");
```

а поток случайных чисел таким образом:

```
Stream<Double> randoms = Stream.generate(Math::random);
```

А для получения бесконечных последовательностей вроде **0 1 2 3 ...** служит метод `iterate()`. Этот метод принимает начальное значение и функцию (а формально — объект функционального интерфейса `UnaryOperator<T>`) и повторно применяет функцию к предыдущему результату, как показано в следующем примере кода:

```
Stream<BigInteger> integers =
      Stream.iterate(BigInteger.ZERO, n -> n.add(BigInteger.ONE));
```

Первым элементом такой последовательности является начальное значение `BigInteger.ZERO`; вторым ее элементом — значение, получаемое в результате вызова функции `f(seed)`, или **1** (как крупное целочисленное значение); следующим элементом — значение, получаемое в результате вызова функции `f(f(seed))`, или **2** и т.д., где `seed` — начальное значение.



**НА ЗАМЕТКУ.** В прикладном программном интерфейсе Java API имеется целый ряд методов, возвращающих потоки данных. Так, в классе **`Pattern`** имеется метод **`splitAsStream()`**, разделяющий последовательность символов типа **`CharSequence`** по регулярному выражению. Например, для разделения символьной строки на отдельные слова можно воспользоваться следующим оператором:

```
Stream<String> words = Pattern.compile("\\PL+").splitAsStream(contents);
```

А статический метод **`Files.lines()`** возвращает поток данных типа **`Stream`**, содержащий все строки из файла, как показано ниже.

```
try (Stream<String> lines = Files.lines(path)) {
 Обработать строки
}
```

## 8.3. Методы `filter()`, `map()` и `flatMap()`

В результате преобразования потока данных получается другой поток данных, элементы которого являются производными от элементов исходного потока. Ранее демонстрировалось преобразование методом `filter()`, в результате которого получается новый поток данных с элементами, удовлетворяющими определенному условию. А в приведенном ниже примере кода поток символьных строк преобразуется в другой поток, содержащий только длинные слова. В качестве аргумента метода `filter()` указывается объект типа `Predicate<T>`, т.е. функция, преобразующая тип **`T`** в логический тип `boolean`.

```
List<String> words = ...;
Stream<String> longWords = words.stream().filter(w -> w.length() > 12);
```

Нередко значения в потоке данных требуется каким-то образом преобразовать. Для этой цели можно воспользоваться методом `map()`, передав ему функцию, которая и выполняет нужное преобразование. Например, буквы во всех словах можно сделать строчными следующим образом:

```
Stream<String> lowercaseWords = words.stream().map(String::toLowerCase);
```

В данном примере методу `map()` была передана ссылка на метод. Но вместо нее нередко передается *лямбда-выражение*, как показано ниже. Получающийся в итоге поток данных содержит первую букву каждого слова.

```
Stream<String> firstLetters = words.stream().map(s -> s.substring(0, 1));
```

При вызове метода `map()` передаваемая ему функция применяется к каждому элементу потока данных, в результате чего образуется новый поток данных с полученными результатами. А теперь допустим, что имеется метод, возвращающий не одно значение, а поток значений, как показано ниже. Например, в результате вызова `letters("boat")` образуется поток данных `["b", "o", "a", "t"]`.

```
public static Stream<String> letters(String s) {
  List<String> result = new ArrayList<>();
  for (int i = 0; i < s.length(); i++)
    result.add(s.substring(i, i + 1));
  return result.stream();
}
```



**НА ЗАМЕТКУ.** Приведенный выше метод **`letters()`** можно реализовать намного изящнее с помощью метода **`IntStream.range()`**, рассматриваемого далее, в разделе 8.13. См. также упражнение 5 в конце этой главы.

Допустим, что метод `letters()` передается методу `map()` для преобразования потока символьных строк следующим образом:

```
Stream<Stream<String>> result = words.stream().map(w -> letters(w));
```

В итоге получится поток потоков символьных строк вроде `[... ["y", "o", "u", "r"], ["b", "o", "a", "t"], ...]`. Чтобы свести его к потоку букв `[... "y", "o", "u", "r", "b", "o", "a", "t", ...]`, вместо метода `map()` следует вызвать метод `flatMap()` таким образом:

```
Stream<String> flatResult = words.stream().flatMap(w -> letters(w))
    // Вызывает метод letters() для каждого слова и сводит результаты
```



**НА ЗАМЕТКУ.** Аналогичный метод **`flatMap()`** можно обнаружить и в других классах, а не в тех, что представляют потоки данных. Это общий принцип вычислительной техники. Допустим, что имеется обобщенный тип `G` (например, **`Stream`**) и функции **`f()`** и **`g()`**, преобразующие некоторый тип `T` в тип `G<U>`, а тип `U` — в тип `G<V>` соответственно. В таком случае эти функции можно составить

вместе, используя метод **flatMap()**, т.е. применить сначала функцию **f()**, а затем функцию **g()**. В этом состоит главная идея теории *монад*. Впрочем, метод **flatMap()** можно применять, и не зная ничего о монадах.

## 8.4. Извлечение и соединение потоков данных

В результате вызова *поток*.limit(n) возвращается поток данных, оканчивающийся после n элементов или по завершении исходного потока данных, если тот короче. Метод limit() особенно удобен для ограничения бесконечных потоков данных до определенной длины. Так, в следующей строке кода получается поток данных, состоящий из 100 произвольных чисел:

```
Stream<Double> randoms = Stream.generate(Math::random).limit(100);
```

А в результате вызова *поток*.skip(n) происходит совершенно противоположное: отбрасываются первые n элементов. Если вернуться к рассмотренному ранее примеру чтения текста книги, то в силу особенностей работы метода split() первым элементом потока данных оказывается нежелательная пустая строка. От нее можно избавиться, вызвав метод skip() следующим образом:

```
Stream<String> words = Stream.of(contents.split("\\PL+")).skip(1);
```

Два потока данных можно соединить вместе с помощью статического метода concat() из интерфейса Stream, как показано ниже. Разумеется, первый из этих потоков не должен быть бесконечным, иначе второй поток вообще не сможет соединиться с ним.

```
Stream<String> combined = Stream.concat(
letters("Hello"), letters("World"));
  // Получается следующий поток данных:
  // ["H", "e", "l", "l", "o", "W", "o", "r", "l", "d"]
```

## 8.5. Другие потоковые преобразования

Метод distinct() возвращает поток данных, получающий свои элементы из исходного потока данных в том же самом порядке, за исключением того, что дубликаты в нем подавляются. Дубликаты не должны быть смежными, как показано ниже.

```
Stream<String> uniqueWords =
  Stream.of("merrily", "merrily", "merrily", "gently").distinct();
  // В итоге возвращается только одна строка "merrily"
```

Для сортировки потоков данных имеется несколько вариантов метода sorted(). Один из них служит для обработки потоков данных, состоящих из элементов типа Comparable, а другой принимает в качестве параметра компаратор типа Comparator. Так, в следующем примере кода символьные строки сортируются таким образом, чтобы первой в потоке данных следовала самая длинная строка:

```
Stream<String> longestFirst =
  words.stream().sorted(Comparator.comparing(String::length).reversed());
```

Как и во всех остальных потоковых преобразованиях, метод `sorted()` выдает новый поток данных, элементы которого происходят из исходного потока и располагаются в отсортированном порядке. Разумеется, коллекцию можно отсортировать, не прибегая к потокам данных. Метод `sorted()` удобно применять в том случае, если процесс сортировки является частью поточного конвейера.

И наконец, метод `peek()` выдает другой поток данных с теми же самыми элементами, что и у исходного потока, но передаваемая ему функция вызывается всякий раз, когда извлекается элемент. Это удобно для целей отладки, как показано ниже.

```
Object[] powers = Stream.iterate(1.0, p -> p * 2)
    .peek(e -> System.out.println("Fetching " + e))
    .limit(20).toArray();
```

Сообщение выводится в тот момент, когда элемент доступен в потоке данных. Подобным образом можно проверить, что бесконечный поток данных, возвращаемый методом `iterate()`, обрабатывается по требованию. Для целей отладки в методе `peek()` можно вызвать метод, в котором устанавливается точка прерывания.

## 8.6. Простые методы сведения

А теперь, когда было показано, каким образом осуществляется создание и преобразование потоков данных, мы наконец-то добрались до самого главного — получения ответов на запросы данных из потоков. В этом разделе рассматриваются так называемые *методы сведения*. Они выполняют *оконечные операции*, сводя поток данных к непотоковому значению, которое может быть далее использовано в программе. Ранее уже демонстрировался простой метод сведения `count()`, возвращающий количество элементов в потоке данных.

К числу других простых методов сведения относятся методы `max()` и `min()`, возвращающие наибольшее и наименьшее значения соответственно. Но не все так просто, поскольку эти методы на самом деле возвращают значение типа `Optional<T>`, которое заключает в себе ответ на запрос данных из потока или обозначает, что запрашиваемые данные отсутствуют, поскольку поток оказался пустым. Раньше в подобных случаях возвращалось пустое значение `null`. Но это могло привести к исключениям в связи с пустыми указателями в не полностью протестированной программе. Тип `Optional` удобнее для обозначения отсутствующего возвращаемого значения. Более подробно тип `Optional` рассматривается в следующем разделе. Ниже показано, как получить максимальное значение из потока данных.

```
Optional<String> largest = words.max(String::compareToIgnoreCase);
System.out.println("largest: " + largest.getOrElse(""));
```

Метод `findFirst()` возвращает первое значение из непустой коллекции. Зачастую он применяется вместе с методом `filter()`. Так, в следующем примере кода обнаруживается первое слово, начинающееся с буквы `Q`:

```
Optional<String> startsWithQ =
    words.filter(s -> s.startsWith("Q")).findFirst();
```

Если же требуется *любое* совпадение, а не только первое, то следует воспользоваться методом `findAny()`, как показано ниже. Это оказывается эффективным при распараллеливании потока данных, поскольку поток может известить о любом обнаруженном в нем совпадении, вместо того чтобы ограничиваться только первым совпадением.

```
Optional<String> startsWithQ =
    words.parallel().filter(s -> s.startsWith("Q")).findAny();
```

Если же требуется только выяснить, имеется *ли* вообще совпадение, то следует воспользоваться методом `anyMatch()`, как показано ниже. Этот метод принимает предикатный аргумент, и поэтому ему не требуется метод `filter()`.

```
boolean aWordStartsWithQ =
   words.parallel().anyMatch(s -> s.startsWith("Q"));
```

Имеются также методы `allMatch()` и `noneMatch()`, возвращающие логическое значение `true`, если с предикатом совпадают все элементы в потоке данных или не совпадает ни один из его элементов соответственно. Эти методы также выгодно выполнять в параллельном режиме.

# 8.7. Тип `Optional`

Объект типа `Optional<T>` служит оболочкой для объекта обобщенного типа `T` или же ни для одного из объектов. В первом случае считается, что значение *присутствует*. Тип `Optional<T>` служит в качестве более надежной альтернативы ссылке на обобщенный тип `T`, которая делается на объект или оказывается пустой. Но этот тип надежнее, если правильно им пользоваться. В следующем разделе поясняется, как это делается.

## 8.7.1. Как обращаться с необязательными значениями

Для эффективного применения типа `Optional` самое главное — выбрать метод, который возвращает *альтернативный вариант*, если значение отсутствует, или *употребляет значение*, если только оно присутствует. Рассмотрим первую методику обращения с необязательными значениями. Нередко имеется значение, возможно, пустая строка `""`, которое требуется использовать по умолчанию в отсутствие совпадения:

```
String result = optionalString.orElse("");
  // Заключенная в оболочку строка, а в ее отсутствие - пустая строка ""
```

Кроме того, можно вызвать функцию для вычисления значения по умолчанию следующим образом:

```
String result = optionalString.orElseGet(() ->
                System.getProperty("user.dir"));
    // Функция вызывается только по мере надобности
```

С другой стороны, в отсутствие значения можно сгенерировать исключение таким образом:

```
String result = optionalString.orElseThrow(IllegalStateException::new);
  // предоставить метод, который выдает объект исключения
```

В приведенных выше примерах было показано, как получить альтернативный вариант, если значение отсутствует. Другая методика обращения с необязательными значениями состоит в том, чтобы употребить значение, если только оно присутствует.

Метод `ifPresent()` принимает функцию в качестве аргумента, как показано ниже. Если необязательное значение существует, оно передается данной функции. В противном случае ничего не происходит.

```
optionalValue.ifPresent(v -> Обработать v);
```

Так, если в множество требуется ввести значение, при условии, что оно существует, достаточно сделать следующий вызов:

```
optionalValue.ifPresent(v -> results.add(v));
```

или просто

```
optionalValue.ifPresent(results::add);
```

В результате вызова метода `ifPresent()` передаваемая ему функция никакого значения не возвращает. Если же требуется обработать результат выполнения функции, следует вызвать метод `map()`, как показано ниже. В итоге переменная `added` будет содержать одно из следующих трех значений: логическое значение `true` или `false`, заключенное в оболочку типа `Optional`, если необязательно значение `optionalValue` присутствовало, а иначе — пустое значение типа `Optional`.

```
Optional<Boolean> added = optionalValue.map(results::add);
```



**НА ЗАМЕТКУ.** Метод **`map()`** служит аналогом метода **`map()`** из интерфейса **`Stream`**, упоминавшегося в разделе 8.3. Необязательное значение можно рассматривать в качестве потока данных нулевой или единичной длины. Получаемый в итоге результат также имеет нулевую или единичную длину. И в последнем случае применялась функция.

## 8.7.2. Как не следует обращаться с необязательными значениями

Если необязательные значения типа `Optional` не применяются правильно, то они не дают никаких преимуществ по сравнению с прежним подходом, предоставлявшим выбор между чем-то существующим или несуществующим, т.е. пустым (`null`). Метод `get()` получает заключенный в оболочку элемент значения типа `Optional`, если это значение существует, а иначе — генерирует исключение типа `NoSuchElementException`. Таким образом, следующий фрагмент кода:

```
Optional<T> optionalValue = ...;
optionalValue.get().someMethod()
```

не надежнее, чем такой код:

```
T value = ...;
value.someMethod();
```

Метод `isPresent()` извещает, содержит ли значение объект типа `Optional<T>`. Но выражение

```
if (optionalValue.isPresent()) optionalValue.get().someMethod();
```

не проще, чем выражение

```
if (value != null) value.someMethod();
```

### 8.7.3. Формирование необязательных значений

До сих пор обсуждалось, как употреблять объект типа `Optional`, созданный кем-то другим. Если же требуется написать метод, создающий объект типа `Optional`, то для этой цели имеется несколько статических методов. В приведенном ниже примере демонстрируется применение двух таких методов: `Optional.of(result)` и `Optional.empty()`.

```
public static Optional<Double> inverse(Double x) {
  return x == 0 ? Optional.empty() : Optional.of(1 / x);
}
```

Метод `ofNullable()` служит в качестве моста между возможными пустыми (`null`) и необязательными значениями. Так, при вызове метода `Optional.ofNullable(obj)` возвращается результат вызова метода `Optional.of(obj)`, если объект `obj` не пустой, а иначе — результат вызова метода `Optional.empty()`.

### 8.7.4. Сочетание функций необязательных значений с методом `flatMap()`

Допустим, что имеется метод `f()`, возвращающий объект типа `Optional<T>`, а у целевого типа `T` — метод `g()`, возвращающий объект типа `Optional<U>`. Если бы это были обычные методы, их можно было бы составить в вызов `s.f().g()`. Но такое сочетание не годится, поскольку результат вызова `s.f()` относится к типу `Optional<T>`, а не к типу `T`. Вместо этого нужно сделать следующий вызов:

```
Optional<U> result = s.f().flatMap(T::g);
```

Если объект, получаемый в результате вызова `s.f()`, присутствует, то к нему применяется метод `g()`. В противном случае возвращается пустой объект типа `Optional<U>`.

Очевидно, что данный процесс можно повторить, если имеются другие методы или лямбда-выражения, возвращающие необязательные значения типа `Optional`. В таком случае из них можно составить конвейер, связав их вызовы в цепочку с методом `flatMap()`, который будет успешно завершен, если завершатся все остальные части конвейера.

В качестве примера рассмотрим надежный метод inverse() из предыдущего раздела. Допустим, что имеется также следующий надежный метод для извлечения квадратного корня:

```
public static Optional<Double> squareRoot(Double x) {
  return x < 0 ? Optional.empty() : Optional.of(Math.sqrt(x));
}
```

В таком случае извлечь квадратный корень из значения, возвращаемого методом inverse(), можно следующим образом:

```
Optional<Double> result = inverse(x).flatMap(MyMath::squareRoot);
```

или таким способом, если он предпочтительнее:

```
Optional<Double> result =
    Optional.of(-4.0).flatMap(Demo::inverse).flatMap(Demo::squareRoot);
```

Если метод inverse() или squareRoot() возвращает результат вызова метода Optional.empty(), то конечный результат окажется пустым.



**НА ЗАМЕТКУ.** Как было показано в разделе 8.3, метод **flatMap()** из интерфейса **Stream** служит для составления двух других методов, получающих потоки данных, сводя их в результирующий поток потоков. Аналогичным образом действует и метод **Optional.flatMap()**, если необязательное значение интерпретируется как поток данных нулевой или единичной длины.

## 8.8. Накопление результатов

По завершении обработки потока данных нередко требуется просмотреть полученные результаты. С этой целью можно вызвать метод iterate(), предоставляющий устаревший итератор, которым можно воспользоваться для обхода элементов. С другой стороны, можно вызвать метод forEach(), чтобы применить функцию к каждому элементу следующим образом:

```
stream.forEach(System.out::println);
```

В параллельном потоке данных метод forEach() выполняет обход элементов в произвольном порядке. Если же их требуется обработать в потоковом порядке, то следует вызвать метод forEachOrdered(). Разумеется, в этом случае могут быть утрачены некоторые или даже все преимущества параллелизма.

Но чаще всего результаты требуется накапливать в структуре данных. С этой целью можно вызвать метод toArray() и получить элементы из потока данных.

Создать обобщенный массив во время выполнения невозможно, и поэтому в результате вызова stream.toArray() возвращается массив типа Object[]. Если же требуется массив нужного типа, то этому методу следует передать конструктор такого массива, как показано ниже.

```
String[] result = stream.toArray(String[]::new);
  // В результате вызова метода stream.toArray()
  // получается массив типа Object[]
```

Для накопления элементов потока данных с другой целью имеется удобный метод `collect()`, принимающий экземпляр класса, реализующего интерфейс `Collector`. В частности, класс `Collectors` предоставляет немало фабричных методов для наиболее употребительных коллекторов. Так, для накопления потока данных в списке или множестве достаточно сделать один из следующих вызовов:

```
List<String> result = stream.collect(Collectors.toList());
```

или

```
Set<String> result = stream.collect(Collectors.toSet());
```

Если же требуется конкретная разновидность получаемого множества, то нужно сделать следующий вызов:

```
TreeSet<String> result =
        stream.collect(Collectors.toCollection(TreeSet::new));
```

Допустим, что требуется накапливать все символьные строки, сцепляя их. С целью можно сделать следующий вызов:

```
String result = stream.collect(Collectors.joining());
```

А если требуется разделитель элементов, то его можно передать методу `joining()` следующим образом:

```
String result = stream.collect(Collectors.joining(", "));
```

Если поток данных содержит объекты, отличающиеся от символьных строк, их нужно сначала преобразовать в символьные строки:

```
String result =
    stream.map(Object::toString).collect(Collectors.joining(", "));
```

Если результаты обработки потока данных требуется свести к сумме, среднему, максимуму или минимуму, воспользуйтесь методами типа `summarizing(Int|Long|Double)`. Эти методы принимают функцию, преобразующую потоковые объекты в число и возвращающую результат типа `(Int|Long|Double)SummaryStatistics`, одновременно вычисляя сумму, среднее, максимум и минимум, как показано ниже.

```
IntSummaryStatistics summary = stream.collect(
    Collectors.summarizingInt(String::length));
double averageWordLength = summary.getAverage();
double maxWordLength = summary.getMax();
```

## 8.9. Накопление результатов в отображениях

Допустим, что имеется поток данных типа `Stream<Person>` и его элементы требуется накапливать в отображении, чтобы в дальнейшем искать людей по их идентификационному номеру. Для этой цели служит метод `Collectors.toMap()`,

принимающий в качестве двух своих аргументов функции, чтобы получить ключи и значения из отображения, как показано в следующем примере кода:

```
Map<Integer, String> idToName = people.collect(
    Collectors.toMap(Person::getId, Person::getName));
```

В общем случае, когда значения должны быть конкретными элементами, в качестве второго аргумента данному методу предоставляется функция `Function.identity()` следующим образом:

```
Map<Integer, Person> idToPerson = people.collect(
    Collectors.toMap(Person::getId, Function.identity()));
```

Если же одному и тому же ключу соответствует больше одного элемента, то возникает конфликт, и коллектор генерирует исключение типа `IllegalStateException`. Это поведение можно изменить, предоставив данному методу в качестве третьего аргумента функцию, разрешающую подобный конфликт и определяющую значение по заданному ключу, исходя из существующего или нового значения. Такая функция может возвратить существующее значение, новое значение или и то и другое.

В приведенном ниже примере создается отображение, содержащее региональные настройки для каждого языка в виде ключа, обозначающего название языка в региональных настройках по умолчанию (например, `"German"`), и значения, обозначающего его локализованное название (например, `"Deutsch"`). В данном примере не учитывается, что один и тот же язык может встретиться дважды (например, немецкий в Германии и Швейцарии), и поэтому в отображении сохраняется лишь первая запись.

```
Stream<Locale> locales = Stream.of(Locale.getAvailableLocales());
Map<String, String> languageNames = locales.collect(
    Collectors.toMap(
        Locale::getDisplayLanguage,
        Locale::getDisplayLanguage,
        (existingValue, newValue) -> existingValue));
```



**НА ЗАМЕТКУ.** В этой главе в качестве структуры данных для хранения региональных настроек употребляется класс **`Locale`**. Подробнее об обращении с региональными настройками речь пойдет в главе 13.

Допустим, что требуется выяснить все языки данной страны. Для этой цели понадобится отображение типа `Map<String, Set<String>>`. Например, значением по ключу `"Switzerland"` является множество `[French, German, Italian]`. Сначала для каждого языка сохраняется одноэлементное множество. А всякий раз, когда обнаруживается новый язык заданной страны, образуется объединение из существующего и нового множеств, как показано ниже.

```
Map<String, Set<String>> countryLanguageSets = locales.collect(
    Collectors.toMap(
        Locale::getDisplayCountry,
        l -> Collections.singleton(l.getDisplayLanguage()),
        (a, b) -> { // объединить множества a и b
            Set<String> union = new HashSet<>(a);
```

```
        union.addAll(b);
        return union; }));
```

Более простой способ получения этого отображения будет представлен в следующем разделе. Если же потребуется древовидное отображение типа `TreeMap`, то в качестве четвертого аргумента методу `toMap()` следует предоставить конструктор данного класса. Необходимо также предоставить функцию объединения. Ниже приведен один из примеров из начала этого раздела, переделанный с целью получить отображение типа `TreeMap`.

```
Map<Integer, Person> idToPerson = people.collect(
  Collectors.toMap(
    Person::getId,
    Function.identity(),
    (existingValue, newValue) -> { throw new IllegalStateException(); },
    TreeMap::new));
```



**НА ЗАМЕТКУ.** Каждому из вариантов метода **`toMap()`** соответствует эквивалентный метод **`toConcurrentMap()`**, получающий параллельное отображение. Единое параллельное отображение применяется в процессе параллельного накопления. Если же общее отображение применяется вместе с параллельным потоком данных, то такой способ оказывается более эффективным, чем объединение множеств. Но в таком случае элементы не накапливаются в потоковом порядке, хотя это обычно не имеет особого значения.

## 8.10. Группирование и разделение

В предыдущем разделе было показано, как накапливаются все языки заданной страны. Но этот процесс оказался несколько трудоемким, поскольку для каждого значения из отображения пришлось сначала сформировать одноэлементное множество, а затем указать порядок объединения существующего и нового значений. Очень часто из значений с одинаковыми характеристиками образуются группы, и этот процесс непосредственно поддерживается методом `groupingBy()`.

Рассмотрим задачу группирования региональных настроек по странам. Сначала образуется следующее отображение:

```
Map<String, List<Locale>> countryToLocales = locales.collect(
    Collectors.groupingBy(Locale::getCountry));
```

Функция `Locale::getCountry()` выполняет роль *классификатора* группирования. Затем все региональные настройки можно отыскать по заданному коду страны, как показано в следующем примере кода:

```
List<Locale> swissLocales = countryToLocales.get("CH");
    // получить региональные настройки [it_CH, de_CH, fr_CH]
```



**НА ЗАМЕТКУ.** Как известно, все региональные настройки состоят из кода языка (например, код **`en`** обозначает английский язык) и кода страницы (например, код **`US`** обозначает Соединенные Штаты). Так, региональные настройки **`en_US`** описывают английский язык в Соединенных Штатах, а

региональные настройки **en_IE** — английский язык в Ирландии. Некоторым странам требуется несколько региональных настроек. Например, региональные настройки **ga_IE**, описывающие гэльский язык в Ирландии в дополнение к упомянутым выше региональным настройкам **en_IE**. А для Швейцарии требуются три региональные настройки, как было показано в предыдущем разделе.

Когда функция классификатора оказывается предикатной (т.е. функцией, возвращающей логическое значение типа `boolean`), элементы потока данных разделяются на основной список с элементами, для которых функция возвращает логическое значение `true`, и дополнительный список. В данном случае эффективнее воспользоваться методом `partitioningBy()`, чем методом `groupingBy()`. Так, в следующем примере кода все региональные настройки разделяются на те, что описывают английский язык, и все остальные:

```
Map<Boolean, List<Locale>> englishAndOtherLocales = locales.collect(
    Collectors.partitioningBy(l -> l.getLanguage().equals("en")));
List<Locale>> englishLocales = englishAndOtherLocales.get(true);
```



**НА ЗАМЕТКУ.** Если вызвать метод **groupingByConcurrent()**, то в конечном итоге будет получено параллельное отображение, которое заполняется параллельно, если оно применяется вместе с параллельным потоком данных. В этом отношении данный метод очень похож на метод **toConcurrentMap()**.

## 8.11. Нисходящие коллекторы

Метод `groupingBy()` формирует множество, значениями которого являются списки. Если эти списки требуется каким-то образом обработать, то следует предоставить *нисходящий коллектор*. Так, если вместо списков требуются множества, то можно воспользоваться коллектором `Collectors.toSet()` следующим образом:

```
Map<String, Set<Locale>> countryToLocaleSet = locales.collect(
    groupingBy(Locale::getCountry, toSet()));
```



**НА ЗАМЕТКУ.** В данном и последующих примерах из этого раздела предполагается статический импорт **java.util.stream.Collectors.***, чтобы упростить выражения и сделать их более удобочитаемыми.

Для сведения сгруппированных элементов к числам предоставляется ряд следующих коллекторов:

- **counting()** — производит подсчет накопленных элементов. Так, в следующем примере кода подчитывается количество региональных настроек для каждой страны:

```
Map<String, Long> countryToLocaleCounts = locales.collect(
    groupingBy(Locale::getCountry, counting()));
```

- **summing(Int|Long|Double)** — принимает в качестве аргумента функцию, применяет ее к элементам нисходящего потока данных и получает их сумму. Так, в следующем примере кода вычисляется суммарное население каждого штата из потока городов:

```
Map<String, Integer> stateToCityPopulation = cities.collect(
groupingBy(City::getState, summingInt(City::getPopulation)));
```

- **maxBy()** и **minBy()** — принимают в качестве аргумента компаратор и получают максимальный и минимальные элементы из нисходящего потока данных. Так, в следующем примере кода получается самый крупный город в каждом штате:

```
Map<String, City> stateToLargestCity = cities.collect(
    groupingBy(City::getState,
        maxBy(Comparator.comparing(City::getPopulation))));
```

- **mapping()** — применяет функцию к результатам, полученным из нисходящего потока данных, но для обработки результатов ему требуется еще один коллектор. Так, в следующем примере кода города группируются по штатам:

```
Map<String, Optional<String>> stateToLongestCityName = cities.collect(
    groupingBy(City::getState,
        mapping(City::getName,
            maxBy(Comparator.comparing(String::length)))));
```

- В каждом штате получаются названия городов, которые сводятся по максимальной длине.

Метод `mapping()` позволяет изящнее решить задачу из предыдущего раздела — собрать все языки, употребляемые в стране. В предыдущем разделе вместо метода `groupingBy()` применялся метод `toMap()`. А в приведенном ниже решении отпадает необходимость объединять отдельные множества.

```
Map<String, Set<String>> countryToLanguages = locales.collect(
    groupingBy(Locale::getDisplayCountry,
        mapping(Locale::getDisplayLanguage,
            toSet())));
```

Если функция группирования или отображения возвращает тип `int`, `long` или `double`, элементы можно накопить в объекте суммарной статистики, как пояснялось в разделе 8.8. Ниже показано, как это делается. А затем из объектов суммарной статистики каждой группы можно получить суммарное, подсчитанное, среднее, минимальное и максимальное значения функции.

```
Map<String, IntSummaryStatistics> stateToCityPopulationSummary =
    cities.collect(
        groupingBy(City::getState,
            summarizingInt(City::getPopulation)));
```



**НА ЗАМЕТКУ.** Имеются три варианта метода **reducing()**, выполняющие общие операции сведения, описываемые в следующем разделе.

Коллекторы можно эффективно сочетать вместе, но в итоге получаются весьма запутанные выражения. Поэтому их лучше всего использовать вместе с методом `groupingBy()` или `partitioningBy()` для обработки значений, преобразуемых из нисходящего потока данных. В противном случае непосредственно в потоках данных просто применяются такие методы, как `map()`, `reduce()`, `count()`, `max()` или `min()`.

## 8.12. Операции сведения

Метод `reduce()` реализует общий механизм для вычисления значения из потока данных. В простейшей форме он принимает двоичную функцию и применяет ее, начиная с первых двух элементов потока данных. Этот механизм проще всего пояснить на примере функции суммирования, как показано ниже.

```
List<Integer> values = ...;
Optional<Integer> sum = values.stream().reduce((x, y) -> x + y);
```

В данном примере метод `reduce()` вычисляет сумму $\mathbf{v}_0 + \mathbf{v}_1 + \mathbf{v}_2 + \ldots$, где $\mathbf{v}_i$ — элементы потока данных. Этот метод возвращает необязательное значение типа `Optional`, поскольку достоверный результат недостижим, если поток данных пуст.



**НА ЗАМЕТКУ.** В данном примере можно сделать вызов **`reduce(Integer::sum)`** вместо вызова **`reduce((x, y) -> x + y)`**.

В общем, если метод `reduce()` выполняет операцию сведения ***op***, то она дает результат $\mathbf{v}_0$ ***op*** $\mathbf{v}_1$ ***op*** $\mathbf{v}_2$ ***op*** $\ldots$, где $\mathbf{v}_i$ ***op*** $\mathbf{v}_{i+1}$ обозначает вызов функции ***op***$(\mathbf{v}_i, \mathbf{v}_{i+1})$. Эта операция должна быть *ассоциативной*, т.е. порядок сочетания ее элементов значения не имеет. В математическом обозначении операция $(x\ op\ y)\ op\ z$ должна быть равнозначна операции $x\ op\ (y\ op\ z)$. Это дает возможность выполнять эффективное сведение в параллельных потоках данных.

Практическую пользу могут принести многие ассоциативные операции, в том числе сложение, умножение, сцепление символьных строк, получение максимума и минимума, объединение и пересечение множеств. Примером операции, которая не является ассоциативной, служит вычитание. Так, $(6 - 3) - 2 \neq 6 - (3 - 2)$.

Нередко имеется *тождественный элемент* ***e*** вроде ***e op x = x***, и он может быть использован в качестве отправной точки для вычисления. Например, `0` является тождественным элементом операции сложения. Ниже приведена вторая форма вызова метода `reduce()`. Тождественное значение возвращается в том случае, если поток данных пуст и больше не нужно обращаться к классу `Optional`.

```
List<Integer> values = ...;
Integer sum = values.stream().reduce(0, (x, y) -> x + y)
  // Вычисляет результат 0 + v0 + v1 + v2 + . . .
```

А теперь допустим, что имеется поток объектов и требуется получить сумму некоторых свойств, например, длину всех символьных строк в потоке. Для этой цели не годится простая форма метода `reduce()`, поскольку в ней требуется функция `(T, T) -> T` с одинаковыми типами аргументов и возвращаемого результата. Но в данном

случае имеются два разных типа: `String` — для элементов потока данных и `int` — для накапливаемого результата. На этот случай имеется отдельная форма вызова метода `reduce()`.

Прежде всего нужно предоставить функцию накопления `(total, word) -> total + word.length()`. Эта функция вызывается повторно, образуя сумму нарастающим итогом. Но если вычисление этой суммы распараллелено, то оно разделяется на несколько параллельных вычислений, результаты которых должны быть объединены. Для этой цели предоставляется вторая функция. Ниже приведена полная форма вызова метода `reduce()` в данном случае.

```
int result = words.reduce(0,
    (total, word) -> total + word.length(),
    (total1, total2) -> total1 + total2);
```



**НА ЗАМЕТКУ.** На практике методом **`reduce()`** приходится пользоваться нечасто. Ведь намного проще преобразовать исходный поток символьных строк в поток чисел и воспользоваться одним из методов для вычисления суммы, максимума или минимума. (Подробнее потоки чисел рассматриваются далее, в разделе 8.13.) В данном конкретном случае можно было бы сделать вызов **`words.mapToInt(String::length).sum()`**. Это было бы проще и эффективнее, поскольку не потребовало бы упаковки.



**НА ЗАМЕТКУ.** Иногда метод **`reduce()`** оказывается недостаточно обобщенным. Допустим, что требуется накопить результаты в множестве типа **`BitSet`**. Если распараллелить эту коллекцию, то разместить ее элементы в одном множестве типа **`BitSet`** не удастся, поскольку объект типа **`BitSet`** не является потокобезопасным. Именно поэтому нельзя воспользоваться методом **`reduce()`**. Каждый сегмент исходной коллекции должен начинаться со своего пустого множества, а методу **`reduce()`** можно предоставить только одно тождественное значение. В таком случае следует воспользоваться методом **`collect()`**, который принимает следующие аргументы.

1. *Поставщик* для получения новых экземпляров целевого объекта. Например, конструктор для построения хеш-множества.
2. *Накопитель*, вводящий элемент в целевой объект. Например, метод **`add()`**.
3. *Объединитель*, соединяющий два объекта в один. Например, метод **`addAll()`**.

Ниже показано, каким образом метод **`collect()`** вызывается для множества битов.

```
BitSet result = stream.collect(BitSet::new, BitSet::set, BitSet::or);
```

## 8.13. Потоки данных примитивных типов

До сих пор целочисленные значения накапливались в потоке данных типа `Stream<Integer>`, несмотря на то, что заключать каждое целочисленное значение в объект-оболочку совершенно неэффективно. Это же относится и к другим примитивным типам данных `double`, `float`, `long`, `short`, `char`, `byte` и `boolean`. В библиотеке потоков данных имеются специализированные классы `IntStream`, `LongStream` и `DoubleStream`, позволяющие сохранять значения примитивных типов непосредственно, не прибегая к помощи оболочек. Так, если требуется сохранить значения типа

short, char, byte и boolean, достаточно воспользоваться классом IntStream, а для хранения значений типа float — классом DoubleStream.

Чтобы создать поток данных типа IntStream, достаточно вызвать методы IntStream.of() и Arrays.stream() следующим образом:

```
IntStream stream = IntStream.of(1, 1, 2, 3, 5);
stream = Arrays.stream(values, from, to);
    // массив values относится к типу int[]
```

К потокам данных примитивных типов, как и к потокам объектов, можно применять статические методы generate() и iterate(). Кроме того, в классах IntStream и LongStream имеются статические методы range() и rangeClosed(), генерирующие диапазоны целочисленных значений с единичным шагом, как показано ниже.

```
IntStream zeroToNinetyNine = IntStream.range(0, 100);
    // Верхний предел исключительно
IntStream zeroToHundred = IntStream.rangeClosed(0, 100);
    // Верхний предел включительно
```

В интерфейсе CharSequence имеются методы codePoints() и chars(), получающие поток типа IntStream кодов символов в Юникоде или кодовых единиц в кодировке UTF-16. (Подробнее о кодировках символов см. в главе 1.) Ниже приведен пример применения метода codePoints().

```
String sentence = "\uD835\uDD46 is the set of octonions.";
    // \uD835\uDD46 — это кодировка UTF-16 знака 𝕆, обозначающего
    // октонионы в Юникоде (U+1D546)

IntStream codes = sentence.codePoints();
    // Поток шестнадцатеричных значений 1D546 20 69 73 20 . . .
```

Поток объектов можно преобразовать в поток данных примитивных типов с помощью методов mapToInt(), mapToLong(() или mapToDouble(). Так, если имеется поток символьных строк и их длины требуется обработать как целочисленные значения, это можно сделать и средствами класса IntStream следующим образом:

```
Stream<String> words = ...;
IntStream lengths = words.mapToInt(String::length);
```

Чтобы преобразовать поток данных примитивного типа в поток объектов, достаточно воспользоваться методом boxed() следующим образом:

```
Stream<Integer> integers = IntStream.range(0, 100).boxed();
```

Как правило, методы для потоков данных примитивных типов аналогичны методам для потоков объектов. Ниже перечислены наиболее существенные их отличия.

- Методы типа toArray возвращают массивы примитивных типов.
- Методы, возвращающие результат необязательного типа, возвращают значение типа OptionalInt, OptionalLong или OptionalDouble. Классы этих типов аналогичны классу Optional, но у них имеются методы getAsInt(), getAsLong() и getAsDouble() вместо метода get().

- Имеются методы `sum()`, `average()`, `max()` и `min()`, возвращающие сумму, среднее, максимум и минимум соответственно. Эти методы не определены для потоков объектов.
- Метод `summaryStatistics()` возвращает объект типа `IntSummaryStatistics`, `LongSummaryStatistics` или `DoubleSummaryStatistics`, способный одновременно сообщать о сумме, среднем, максимуме и минимуме в потоке данных.



**НА ЗАМЕТКУ.** В классе **`Random`** имеются методы **`ints()`**, **`longs()`** и **`doubles()`**, возвращающие потоки данных примитивных типов, состоящие из случайных чисел.

## 8.14. Параллельные потоки данных

Потоки данных упрощают распараллеливание групповых операций. Этот процесс происходит в основном автоматически, но требует соблюдения немногих правил. Прежде всего, нужно иметь в своем распоряжении параллельный поток данных. Получить параллельный поток данных можно из любой коллекции с помощью метода `Collection.parallelStream()` следующим образом:

```
Stream<String> parallelWords = words.parallelStream();
```

Более того, метод `parallel()` преобразует любой последовательный поток данных в параллельный, как показано ниже. При выполнении оконечного метода поток данных действует в параллельном режиме, и поэтому промежуточные операции в этом потоке распараллеливаются.

```
Stream<String> parallelWords = Stream.of(wordArray).parallel();
```

Если потоковые операции выполняются параллельно, то и получаемый в итоге результат должен быть таким же, как и при последовательном выполнении операций. Очень важно, чтобы эти операции выполнялись *без сохранения состояния* и в произвольном порядке.

Допустим, что требуется подсчитать все короткие слова в потоке символьных строк. В приведенном ниже примере демонстрируется, как не следует решать эту задачу.

```
int[] shortWords = new int[12];
words.parallelStream().forEach(
    s -> { if (s.length() < 12) shortWords[s.length()]++; });
      // ОШИБКА: состояние гонок!
System.out.println(Arrays.toString(shortWords));
```

Приведенный выше код написан очень скверно. Функция, передаваемая методу `forEach()`, выполняется параллельно в нескольких потоках исполнения, в каждом из которых обновляется разделяемый ими общий массив. Как будет показано в главе 10, это классическое *состояние гонок*. Если выполнить данный код многократно, то в

результате каждого его выполнения, вероятнее всего, будет получена совсем другая последовательность подсчитанных коротких слов, причем каждый раз неверная.

В обязанности программиста входит обеспечение надежного выполнения в параллельном режиме функций, передаваемых для распараллеливания операций в потоке данных. Для этого лучше всего избегать изменяемого состояния. В следующем примере кода вычисления можно надежно распараллелить, если сгруппировать символьные строки по длине и подсчитать их:

```
Map<Integer, Long> shortWordCounts =
    words.parallelStream()
        .filter(s -> s.length() < 10)
        .collect(groupingBy(
            String::length,
            counting()));
```

По умолчанию потоки данных, получаемые из упорядоченных коллекций (массивов и списков), диапазонов, генераторов, итераторов или в результате вызова метода `Stream.sorted()`, *упорядочиваются*. Результаты накапливаются в порядке следования исходных элементов и полностью предсказуемы. Если выполнить одни и те же операции дважды, то будут получены совершенно одинаковые результаты.

Упорядочение не исключает эффективное распараллеливание. Например, при вызове `stream.map(fun)` поток данных может быть разбит на *n* сегментов, каждый из которых обрабатывается параллельно. А полученные результаты снова собираются по порядку.

Некоторые операции могут быть распараллелены более эффективно, если требование упорядочения опускается. Вызывая метод `Stream.unordered()`, можно указать, что упорядочение не имеет значения. Это, в частности, выгодно при выполнении операции методом `Stream.distinct()`. В упорядоченном потоке метод `distinct()` сохраняет первый из всех равных элементов. Этим ускоряется распараллеливание, поскольку в потоке исполнения, обрабатывающем отдельный сегмент, неизвестно, какие именно элементы следует отбросить, до тех пор, пока сегмент не будет обработан. Если же допускается сохранить *любой* однозначный элемент, то все сегменты могут быть обработаны параллельно (с помощью общего множества для отслеживания дубликатов).

Если опустить упорядочение, то можно также ускорить выполнение метода `limit()`. Если же требуется обработать *любые* *n* элементов из потока данных и при этом неважно, какие из них будут получены, то с этой целью можно сделать следующий вызов:

```
Stream<String> sample = words.parallelStream().unordered().limit(n);
```

Как обсуждалось в разделе 8.9, объединять отображения невыгодно. Именно поэтому в методе `Collectors.groupingByConcurrent()` используется общее параллельное отображение. Чтобы извлечь выгоду из параллелизма, порядок следования значений в отображении должен быть иным, чем в потоке данных:

```
Map<Integer, List<String>> result = words.parallelStream().collect(
    Collectors.groupingByConcurrent(String::length));
    // Значения не накапливаются в потоковом порядке
```

Разумеется, это не имеет особого значения, если применяется нисходящий коллектор, не зависящий от упорядочения, как в следующем примере кода:

```
Map<Integer, Long> wordCounts =
    words.parallelStream()
        .collect(
            groupingByConcurrent(
                String::length,
                counting()));
```



**ВНИМАНИЕ.** При выполнении потоковой операции очень важно не изменять коллекцию, поддерживающую поток данных, даже если такое изменение и является потокобезопасным. Напомним, что данные в потоках не накапливаются, а всегда находятся в отдельной коллекции. Если попытаться изменить коллекцию, то результат выполнения потоковых операций окажется неопределенным. В документации на комплект JDK такое требование называется *невмешательством*. Оно относится как к последовательным, так и к параллельным потокам данных.

Точнее говоря, коллекцию можно изменять вплоть до момента выполнения оконечной операции, поскольку промежуточные потоковые операции выполняются по требованию. Так, следующий фрагмент кода вполне работоспособен, хотя и не рекомендуется:

```
List<String> wordList = ...;
Stream<String> words = wordList.stream();
wordList.add("END");
long n = words.distinct().count();
```

А приведенный ниже фрагмент кода оказывается неработоспособным.

```
Stream<String> words = wordList.stream();
words.forEach(s -> if (s.length() < 12) wordList.remove(s));
     // ОШИБКА: вмешательство!
```

## Упражнения

1. Убедитесь, что запрос первых пяти длинных слов не требует вызова метода `filter()`, если найдено пятое длинное слово. С этой целью просто организуйте протоколирование вызова каждого метода.
2. Определите разность во времени подсчета длинных слов с помощью методов `parallelStream()` и `stream()`. Вызовите метод `System.currentTimeMillis()` до и после этих методов и выведите разность. Если у вас быстродействующий компьютер, выберите для подсчета длинных слов более длинный документ (например, роман "Война и мир").
3. Допустим, что имеется массив `int[] values = { 1, 4, 9, 16 }`. Каков результат вызова `Stream.of(values)`? Как вместо этого получить поток данных типа `int`?
4. С помощью метода `Stream.iterate()` создайте бесконечный поток случайных чисел, не вызывая метод `Math.random()`, а непосредственно реализуя *линейный конгруэнтный генератор*. Такой генератор начинает действовать с числа, задаваемого выражением $x_0$ = *начальное значение*, а затем производит случайные

числа по формуле $x_{n+1} = (a\ x_n + c)\ \%\ m$, при соответствующих значениях $a$, $c$ и $m$. С этой целью реализуйте метод, принимающий параметры $a$, $c$, $m$ и получающий поток данных `Stream<Long>`. Опробуйте его со следующими параметрами: $a$ = **25214903917**, $c$ = **11** и $m = 2^{48}$.

5. Метод `letters()`, упоминавшийся в разделе 8.3, выглядит несколько неуклюже, поскольку в нем сначала получается списочный массив, а затем он превращается в поток данных. Напишите другой, однострочный вариант этого метода, используя поток данных. Преобразуйте значения типа `int` в пределах от **0** до `s.length() - 1` с помощью подходящего лямбда-выражения.
6. Воспользуйтесь методом `String.codePoints()` для реализации метода, проверяющего, является ли символьная строка словом, состоящим только из букв. (*Подсказка:* воспользуйтесь методом `Character.isAlphabetic()`.) Реализуйте тем же самым способом метод, проверяющий, является ли символьная строка достоверным в Java идентификатором.
7. Преобразовав содержимое файла в поток лексем, выведите список первых 100 лексем, являющихся словами в том смысле, в каком они определены в предыдущем упражнении. Прочитайте содержимое файла снова и выведите список из 10 наиболее часто употребляемых слов, игнорируя регистр букв.
8. Прочитайте слова из файла `/usr/share/dict/words` (или аналогичного списка слов) в поток данных и получите массив всех слов, содержащих пять отдельных гласных.
9. Определите среднюю длину строки в заданном конечном потоке символьных строк.
10. Определите все символьные строки максимальной длины в заданном конечном потоке символьных строк.
11. Допустим, что ваш непосредственный начальник дал вам задание написать метод `public static <T> boolean isFinite(Stream<T> stream)`. Почему это не самая удачная мысль? Все равно напишите этот метод.
12. Напишите метод `public static <T> Stream<T> zip(Stream<T> first, Stream<T> second)`, изменяющий элементы из потоков данных `first` и `second` (или возвращающий пустое значение `null`, если в потоке данных, черед которого настает, исчерпываются элементы).
13. Соедините все элементы в потоках данных `Stream<ArrayList<T>>` и `ArrayList<T>`. Покажите, как добиться этого с помощью каждой из трех форм метода `reduce()`.
14. Организуйте вызов метода `reduce()` таким образом, чтобы вычислить среднее в потоке данных `Stream<Double>`. Почему нельзя просто вычислить сумму и разделить ее на результат, возвращаемый методом `count()`?
15. Найдите 500 простых чисел с 50 десятичными цифрами, используя параллельный поток данных типа `BigInteger` и метод `BigInter.isProbablePrime()`.

Насколько это делается быстрее, чем при использовании последовательного потока данных?

16. Найдите 500 самых длинных слов в романе "Война и мир", используя параллельный поток данных. Насколько это делается быстрее, чем при использовании последовательного потока данных?
17. Каким образом можно исключить получение смежных дубликатов из потока данных? Сможет ли написанный вами метод обрабатывать параллельный поток?